

POS-itive Disambiguation: WSD and the Power of Transformers-based Models combined with POS Taggings

MNLP - Homework 2 Report

Antonello Giorgio

1836529

giorgio.1836529@studenti.uniroma1.it

1 Introduction

In this report, the problem of Word Sense Disambiguation is addressed. Word Sense Disambiguation (WSD) is the process of determining the appropriate sense or meaning of a word within a given context. It is a critical component of NLP, as many words exhibit multiple meanings depending on their context. WSD aims to resolve this ambiguity. My approach always involves a Transformer-based architecture, because Transformers excel at capturing contextual information. Their ability to model complex relationships and adapt to different contexts makes them a powerful tool for disambiguating word senses in texts. Moreover, in order to better enhance the model performances, I focused the experiments on the addition of Part-Of-Speech (POS) embeddings. In the next sections I'm going to show why this choice of implementation and how it affects the experiments results.

2 Data Preparation

2.1 Dataset Description

There are two dataset files:

- A coarse-grained dataset, containing candidates and correct gold homonymy clusters.
- A fine-grained dataset, containing candidates and correct gold WordNet senses.

I decided to use only the coarse-grained dataset, which contains samples structured in this way:

- `idx`: document id and sentence id clusters.
- `instance_ids`: mapping between token based offsets of each instance and its id
- `words`: list of tokenized words
- `lemmas`: list of tokenized and lemmatized words

- `pos_tags`: list of part of speech tags
- `senses`: mapping between token based offsets and gold homonymy clusters
- `candidates`: list of possible homonymy clusters for each instance

2.2 Data Preprocessing

Before feeding the data into the model, since the tokenizer takes non-splitting texts, I first obtained the entire sentences from the lists of words or lemmas, based on a boolean variable. Lemmas capture the core semantic meaning of words, as they represent the base form of a word. This can lead to better generalization when training a transformer model for WSD. Instead of dealing with various inflections of a word, the model focuses on the essential sense distinctions related to lemmas. However, in some cases words may contain more valuable context for disambiguation than lemmas: this led to different experiments with both forms, in order to check which is better for the problem of WSD. For POS tags and senses, I converted them to numbers through vocabularies obtained from the training data, in order to be accepted as model inputs. For the candidates, I used them to build vocabulary masks that isolate only possible senses representations for the target instance.

3 Methodology

3.1 Transformer-based Models

Since I want to use only the encoder part of the Transformer architecture, the choice of models includes BERT (Bidirectional Encoder Representations from Transformers) and RoBERTa, which are very effective for WSD because they learn a lot about word meanings and context during their pre-training. Figure 1 shows that most of the sentences have less than a hundred of words, so there's no

need to truncate sentences while using the transformer model. The output of the transformer will be the average of the last four hidden states.

3.2 Tokenization Process

The tokenization process made the target embeddings production very challenging, because:

- tokenizers usually add special tokens that could vary the target position in the sentence and, consequently, make it more difficult to find the right embeddings in the transformer output.
- a target word (or lemma) can be divided in multiple subtokens, so we need to keep track of multiple embeddings to build the final output.
- words: a target word (or lemma) could consist of more than one actual word (e.g. 'found out'); same problem as in the previous point.

All these issues led to a led to an initial tokenization of the target word, so as to understand how it was being subdivided, and then once the whole sentence was tokenized, the indices related to the positions of the tokens belonging to the target word were obtained: in this way, once the output of the transformer was obtained, the target embeddings could be accessed directly, and, by averaging them together, the final target embedding for the target word was obtained.

3.3 POS Embedding Layer

POS tags can be a very useful resource for WSD, since they provide information about the syntactic role of a word in a sentence. Moreover, with the idea that whenever a word appears with the same POS tag then it will most likely have the same sense, I plotted in Figure 2 the total number of word-pos pairs, the number of word-pos pairs that appear more than once, and the number of word-pos pairs that appear more than once and all have the same sense. In the training data, 53% of word-pos pairs appear more than once, and 84% of these have the same sense for each of their appearances: this confirms the starting idea.

The choices of implementation can be:

- pass them as a further input to the transformer model
- combine their embeddings with the output of the transformer model

I chose the second option, which leads to other three possibilities: after passing POS tags through the POS Embedding Layer,

- concatenate POS and Transformer embeddings.
- sum POS and Transformer embeddings.
- average POS and Transformer embeddings.

I decided to average them, but in future works trying the other approaches could be very interesting.

3.4 Additional Layers

After transformer and POS embedding layers, the output is first passed through a Dropout Layer, to prevent overfitting and improve the generalization of the model, and then through a fully-connected Linear layer, which will produce the logits for the target word. Finally, the candidate masks are applied on the logits to isolate the only possible candidates, and the one with the highest score will be the predicted sense.

4 Experiments

4.1 Experimental Setup

All experiments were done by exploiting pytorch lightning functions for training the model and saving weights and results along epochs: the latters are shown as plots by reading CSV-files obtained during training phase. Since we have a multi-classification problem, Cross Entropy was used as loss function. To update the model weights, Adam optimizer was chosen.

4.2 Results

For the results, the main metric to observe is the accuracy (that's the number of correctly predicted senses for the target words w.r.t. the total number of target words). In order to compare the different approaches regarding input and embedding selection, I decided to execute four different experiments with the same hyperparameters. All of them were done with BERT as Transformer Model and Tokenizer. The first experiment is to test the model with words as input (hyperparameter "use_lemmas" equal to "False," Figure 3): both loss and accuracy have good behavior, having the validation metrics following the trend of the training metrics. Now, let's focus on the model with lemmas

as input (Figure 4): training loss and accuracy immediately reach their minimum and maximum, respectively, while validation loss and accuracy have a slower variation; moreover, unlike validation loss, validation accuracy also starts to deteriorate. An important variation for the performance is the addition of POS tags: both word-based (Figure 5) and lemma-based (Figure 6) models improve, having the valid accuracy (word-based) that reaches new peaks, and train loss/accuracy (lemma-based) with a slower decrease/increase. Finally, in Table 1, valid and test accuracy values are reported for each experiment. The "word_bert" model gives impressive results, while "lemma_bert" model is the worst of all. Its POS-counterpart gives better results, as predicted, but lower than "word_pos_bert", which has a slightly lower valid accuracy than the "word_bert" one, but a higher test accuracy. This means that "word_pos_bert" model is able to generalize more than the other models, and this is the reason why I decided to execute another test with this model (Figure 7). The performances of this new model (with a higher number of epochs and a higher dropout rate) should be more reliable, but actually the valid accuracy does not increase as expected (and so does the test accuracy, Table 1).

5 Conclusion

In this report, the problem of Word Sense Disambiguation (WSD) was approached primarily focusing on leveraging the power of Transformers to capture contextual information effectively. Additionally, the incorporation of Part-Of-Speech (POS) embeddings was explored to enhance model performance. The results showed that the model with words as input achieved impressive accuracy, while the lemma-based model performed the worst. The addition of POS tags improved performance, with the word-based model outperforming the lemma-based one. The "word_pos_bert" model demonstrated strong generalization, as reflected in its test accuracy. Experiments highlighted the effectiveness of Transformer-based models for WSD, especially when combined with POS embeddings. The "word_pos_bert" model showed promise as a robust performer, although future works, such as tuning hyperparameters and experimenting with different variations in combining POS embeddings with the transformer model input/output, are necessary to optimize its performance fully.

6 Tables and Figures

Experiment	Valid Acc	Test Acc
word_bert	0.9031	0.8893
lemma_bert	0.8961	0.8795
word_pos_bert	0.9000	0.8956
lemma_pos_bert	0.8982	0.8902
word_pos_bert_10epochs	0.8995	0.8902

Table 1: Experiments Results.

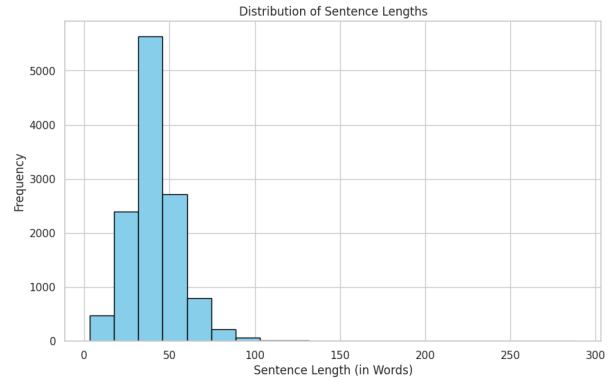


Figure 1: Sentences Lengths Distribution.

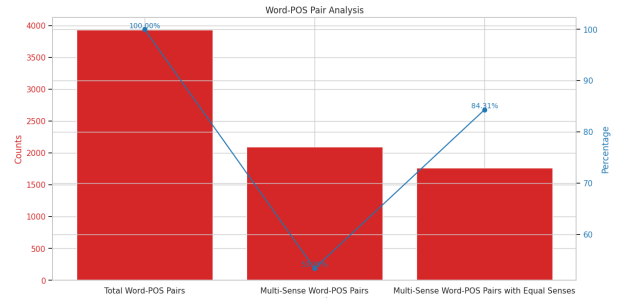


Figure 2: Word/Pos Pair Analysis.

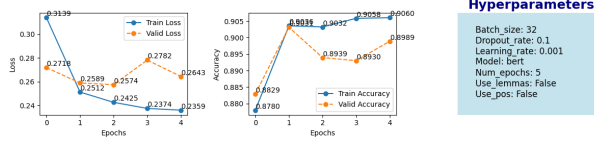


Figure 3: WSD with words as inputs and BERT as transformer model.

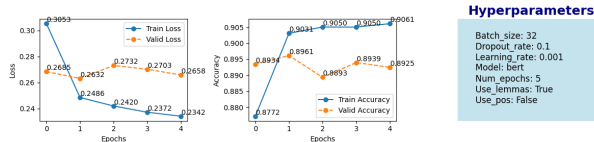


Figure 4: WSD with lemmas as inputs and BERT as transformer model.

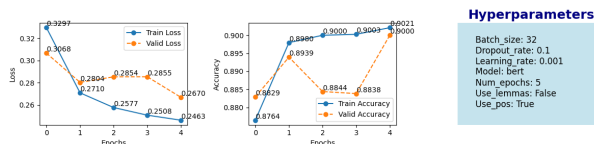


Figure 5: WSD with words as inputs, BERT as transformer model, with POS tags embeddings.

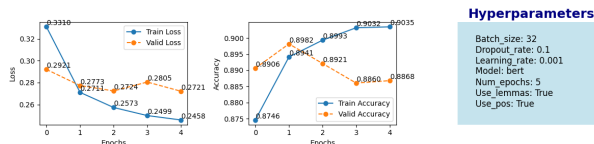


Figure 6: WSD with lemmas as inputs, BERT as transformer model, with POS tags embeddings.

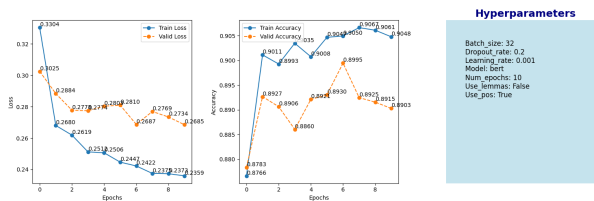


Figure 7: WSD with words as inputs, BERT as transformer model, with POS tags embeddings and different hyperparameters.